

Unit ID: 6788dd0bbc872f1c3f398220

Introduction to Data Structures

Dictionaries in Python

Key-Value Pairs

Understanding Lists

Python Tutorial (list and function)

Tuples in Python

Data structures are a way to organize and store data so that it can be accessed and modified efficiently. They form the backbone of programming, helping developers handle, manage, and analyze data effectively. Python provides a rich set of built-in data structures such as lists, tuples, dictionaries, and sets, which cater to different programming needs. In programming, the way data is structured impacts the efficiency and clarity of the code. For instance, lists are ideal for ordered data, while dictionaries are better suited for key-value pairs. Python's built-in data structures provide flexibility and simplicity, making them a preferred choice for beginners and experts alike. This unit introduces you to these fundamental structures and their applications in real-world scenarios.

Python lists are powerful tools for organizing data. Whether you're creating a to-do list or managing datasets, Python lists offer functionalities such as slicing, indexing, and nesting to simplify your tasks. With methods like `append()`, `pop()`, and `sort()`, lists make it easy to manipulate data as needed.

Tuples provide a reliable way to store data that should remain constant throughout a program. For example, coordinates, dates, and configuration settings are ideal use cases for tuples. Unlike lists, tuples offer performance benefits due to their immutability, making them an efficient choice for read-only data.

Dictionaries store data in the form of key-value pairs, where each key is unique. They are perfect for associating related data, such as a person's name and their phone number. Dictionaries are one of Python's most powerful and flexible data structures. With features like fast lookups and dynamic resizing, dictionaries are ideal for handling structured data. Methods like `get()`, `update()`, and `items()` make it easy to retrieve and manipulate data stored in a dictionary.

Gallery Images

Image not found: 1737022730354-python-list.jpg

PDF Documents

PDF Document 1: 1737022730355-AssignII-PythonStudy.pdf

This PDF document is included in the unit materials.

YouTube Links

Python Tutorial (list and function)

Adopted from: <https://www.w3schools.com/python/default.asp>

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered and unindexed. No duplicate members.
- **Dictionary** is a collection which is unordered, changeable and indexed. No duplicate members.

When choosing a collection type, it is useful to understand the properties of that type. Choosing the right type for a particular data set could mean retention of meaning, and, it could mean an increase in efficiency or security.

List

A list is a collection which is ordered and changeable. In Python lists are written with square brackets.

Create a List:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist)
```

Access Items

You access the list items by referring to the index number

Print the second item of the list:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[1])
```

Negative Indexing

Negative indexing means beginning from the end, -1 refers to the last item, -2 refers to the second last item etc.

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[-1])
```

Range of Indexes

You can specify a range of indexes by specifying where to start and where to end the range.

When specifying a range, the return value will be a new list with the specified items.

Return the third, fourth, and fifth item:

```
thislist = ["apple", "banana", "cherry", "orange",  
            "kiwi", "melon", "mango"]  
print(thislist[2:5])
```

The search will start at index 2 (included) and end at index 5 (not included).

By leaving out the start value, the range will start at the first item:

This example returns the items from the beginning to "orange":

```
thislist = ["apple", "banana", "cherry", "orange",  
            "kiwi", "melon", "mango"]  
print(thislist[:4])
```

By leaving out the end value, the range will go on to the end of the list:

This example returns the items from "cherry" and to the end:

```
thislist = ["apple", "banana", "cherry", "orange",  
            "kiwi", "melon", "mango"]  
print(thislist[2:])
```

Change Item Value

To change the value of a specific item, refer to the index number:

Change the second item:

```
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist)
```

Loop Through a List

You can loop through the list items by using a `for` loop:

Print all items in the list, one by one:

```
thislist = ["apple", "banana", "cherry"]
for x in thislist:
    print(x)
```

Check if Item Exists

To determine if a specified item is present in a list use the `in` keyword:

```
Check if "apple" is present in the list:
thislist = ["apple", "banana", "cherry"]
if "apple" in thislist:
    print("Yes, 'apple' is in the fruits list")
```

List Length

To determine how many items a list has, use the `len()` function:

```
Print the number of items in the list:
thislist = ["apple", "banana", "cherry"]
print(len(thislist))
```

Add Items

To add an item to the end of the list, use the `append()` method:

Using the `append()` method to append an item:

```
thislist = ["apple", "banana", "cherry"]
thislist.append("orange")
print(thislist)
```

To add an item at the specified index, use the `insert()` method:

Insert an item as the second position:

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(1, "orange")
print(thislist)
```

Remove Item

There are several methods to remove items from a list:

The `remove()` method removes the specified item:

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist)
```

The `pop()` method removes the specified index, (or the last item if index is not specified):

```
thislist = ["apple", "banana", "cherry"]
thislist.pop()
print(thislist)
```

The `clear()` method empties the list:

```
thislist = ["apple", "banana", "cherry"]
thislist.clear()
print(thislist)
```

The `del` keyword removes the specified index:

```
thislist = ["apple", "banana", "cherry"]
del thislist[0]
print(thislist)
```

The `del` keyword can also delete the list completely:

```
thislist = ["apple", "banana", "cherry"]
del thislist
```

Copy a List

You cannot copy a list simply by typing `list2 = list1`, because: `list2` will only be a *reference* to `list1`, and changes made in `list1` will automatically also be made in `list2`.

There are ways to make a copy, one way is to use the built-in List method `copy()`.

Make a copy of a list with the `copy()` method:

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy()
print(mylist)
```

Another way to make a copy is to use the built-in method `list()`.

Make a copy of a list with the `list()` method:

```
thislist = ["apple", "banana", "cherry"]
mylist = list(thislist)
print(mylist)
```

Join Two Lists

There are several ways to join, or concatenate, two or more lists in Python.

One of the easiest ways are by using the `+` operator.

Join two list:

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
```

```
list3 = list1 + list2
print(list3)
```

Another way to join two lists are by appending all the items from `list2` into `list1`, one by one:

Append `list2` into `list1`:

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
for x in list2:
    list1.append(x)
print(list1)
```

Or you can use the `extend()` method, which purpose is to add elements from one list to another list:

Use the `extend()` method to add `list2` at the end of `list1`:

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
```

```
list1.extend(list2)
print(list1)
```

List Methods

Python has a set of built-in methods that you can use on lists.

Method	Description
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

Python Functions

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

Creating a Function

In Python a function is defined using the `def` keyword:

```
def my_function():  
    print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

```
def my_function():  
    print("Hello from a function")  
my_function()
```

Arguments

Information can be passed into functions as arguments.

Parameters or Arguments?

The terms *parameter* and *argument* can be used for the same thing: information that are passed into a function.

From a function's perspective:

A parameter is the variable listed inside the parentheses in the function definition.

An argument is the value that is sent to the function when it is called.

Number of Arguments

By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

This function expects 2 arguments, and gets 2 arguments:

```
def my_function(fname, lname):  
    print(fname + " " + lname)  
  
my_function("Emil", "Refsnes")
```

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

```
def my_function(fname):  
    print(fname + " Refsnes")  
  
my_function("Emil")  
my_function("Tobias")  
my_function("Linus")
```

Arguments are often shortened to *args* in Python documentations.

If you try to call the function with 1 or 3 arguments, you will get an error:

This function expects 2 arguments, but gets only 1:

```
def my_function(fname, lname):  
    print(fname + " " + lname)  
  
my_function("Emil")
```

Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a `*` before the parameter name in the function definition.

This way the function will receive a *tuple* of arguments, and can access the items accordingly:

If the number of arguments is unknown, add a `*` before the parameter name:

```
def my_function(*kids):  
    print("The youngest child is " + kids[2])  
  
my_function("Emil", "Tobias", "Linus")
```

Arbitrary Arguments are often shortened to **args* in Python documentations.

Keyword Arguments

You can also send arguments with the *key = value* syntax.

This way the order of the arguments does not matter.

```
def my_function(child3, child2, child1):  
    print("The youngest child is " + child3)
```

```
my_function(child1 = "Emil", child2 = "Tobias",  
            child3 = "Linus")
```

The phrase *Keyword Arguments* are often shortened to *kwargs* in Python documentations.

Arbitrary Keyword Arguments, **kwargs

If you do not know how many keyword arguments that will be passed into your function, add two asterisk: **** before the parameter name in the function definition.

This way the function will receive a *dictionary* of arguments, and can access the items accordingly:

If the number of keyword arguments is unknown, add a double **** before the parameter name:

```
def my_function(**kid):  
    print("His last name is " + kid["lname"])
```

```
my_function(fname = "Tobias", lname = "Refsnes")
```

Arbitrary Kword Arguments are often shortened to ***kwargs* in Python documentations.

Default Parameter Value

The following example shows how to use a default parameter value.

If we call the function without argument, it uses the default value:

```
def my_function(country = "Norway"):  
    print("I am from " + country)
```

```
my_function("Sweden")  
my_function("India")  
my_function()  
my_function("Brazil")
```

Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

```
def my_function(food):  
    for x in food:  
        print(x)
```

```
fruits = ["apple", "banana", "cherry"]  
my_function(fruits)
```

Return Values

To let a function return a value, use the *return* statement:

```
def my_function(x):  
    return 5 * x  
  
print(my_function(3))  
print(my_function(5))  
print(my_function(9))
```

The pass Statement

function definitions cannot be empty, but if you for some reason have a *function* definition with no content, put in the *pass* statement to avoid getting an error.

```
def myfunction():  
    pass
```

Recursion

Python also accepts function recursion, which means a defined function can call itself.

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

The developer should be very careful with recursion as it can be quite easy to slip into writing a function which never terminates, or one that uses excess amounts of memory or processor power. However, when written correctly recursion can be a very efficient and mathematically-elegant approach to programming.

In this example, tri_recursion() is a function that we have defined to call itself ("recurse"). We use the k variable as the data, which decrements (-1) every time we recurse. The recursion ends when the condition is not greater than 0 (i.e. when it is 0).

To a new developer it can take some time to work out how exactly this works, best way to find out is by testing and modifying it.

Recursion Example

```
def tri_recursion(k):
    if(k > 0):
        result = k + tri_recursion(k - 1)
        print(result)
    else:
        result = 0
    return result

print("\n\nRecursion Example Results")
tri_recursion(6)
```

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python –

- Global variables
- Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example –

```
total = 0; # This is global variable.
# Function definition is here
def sum( arg1, arg2 ):
    # Add both the parameters and return
    them."
    total = arg1 + arg2; # total is local
    print "Inside function L total : ", total
    return total;

# Now you can call sum function
sum( 10, 20 );
print "Outside function G total : ", total
```

The Anonymous Lambda Functions

Lambda forms can take any number of arguments but return just one value in the form of an expression

The syntax of *lambda* functions contains only a single statement, Function definition is here

```
sum = lambda arg1, arg2: arg1 + arg2;

# Now you can call sum as a function
print "Value of total : ", sum( 10, 20 )
print "Value of total : ", sum( 20, 20 )
```